CNN Visualization and Fault-Tolerant Convolution

Convolutional Neural Networks (CNNs) are specialized deep learning architectures designed to process grid-structured data such as images. Their core operation—convolution—preserves spatial relationships through learnable filters. This assignment explores:

1. Classical image filtering techniques
2. Kernel size and stride variations
3. Pooling mechanisms
4. Learning filters using gradient descent
5. Feature visualization in deep CNN models
6. Layer-wise filter distribution and duplicate detection
7. Fault-tolerant CNN verification using checksum schemes

The implementation covers both theoretical validation and practical experimentation using PyTorch and TensorFlow/Keras.

Number of Filters Applied to a Particular Input

In a convolutional layer, the number of filters applied to an input equals the number of output channels of that layer.

Input shape = (C_in, H, W)

Kernel shape = (C_out, C_in, k, k)

Then:

Total filters applied = C_out

Output feature maps = C_out

Input shape: [5, 3, 32, 32]
Kernel shape: [8, 3, 3, 3]
Output shape: [5, 8, 30, 30]

Here:

- 3 = Input channels (RGB)
- 8 = Number of filters
- 8 feature maps are generated per image

Thus:

The number of filters applied equals the number of output channels in the convolutional layer.

Single MNIST Image with a Single Filter

MNIST images are grayscale with shape:

(1, 28, 28)

If a single 3×3 filter is applied with stride = 1 and no padding:

Output dimension = 28 − 3 + 1 = 26

Output shape becomes:

(1, 26, 26)

Since only one filter is used:

> One feature map is produced.
>
> The filter extracts one specific spatial feature (e.g., vertical edge).

Thus:

For a single MNIST image with a single filter, exactly one feature map is generated.

Kernel, Stride and Pooling Variations

Kernel sizes tested:

- 5 × 5
- 9 × 9
- 11 × 11

Increasing kernel size increases:

- Receptive field
- Spatial smoothing

- Abstraction level

Larger kernels capture more global structures but reduce output dimensions unless padding is applied.

Stride Variation

Strides applied:

- 1
- 3
- 5
- 7

Stride introduces spatial sparsity in feature extraction.

Pooling Operations

Three pooling methods were applied:

- Max Pooling
- Average Pooling
- Min Pooling

Observations:

Pooling Type	Behavior
Max Pooling	Retains strongest activations
Average Pooling	Smooths regions
Min Pooling	Retains lowest intensity

Pooling reduces spatial dimensions while preserving dominant features.

Gradient Descent Based Filter Learning

A randomly initialized 3×3 filter was trained to approximate a predefined Sobel-like target filter.

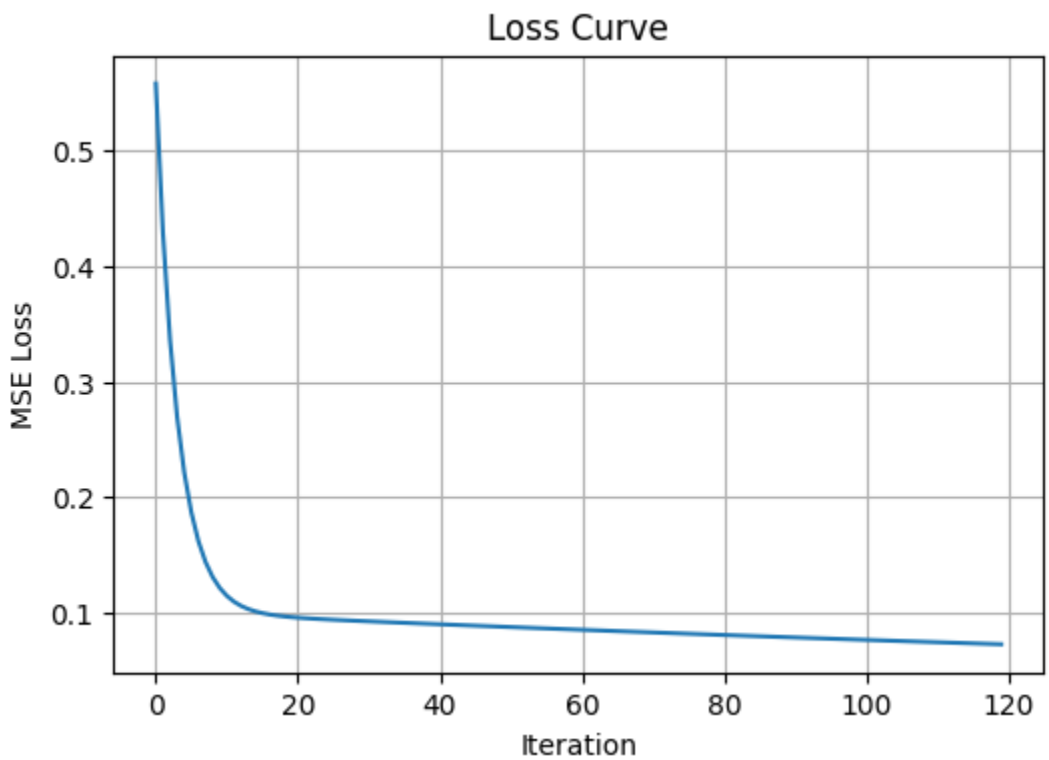
Loss function used:

Mean Squared Error (MSE)

Training process:

1. Forward convolution using target kernel
2. Forward convolution using learned kernel
3. Compute MSE loss
4. Backpropagate gradients
5. Update kernel using SGD

Observation:



- Loss curve decreases steadily.
- Learned kernel converges close to target kernel values.

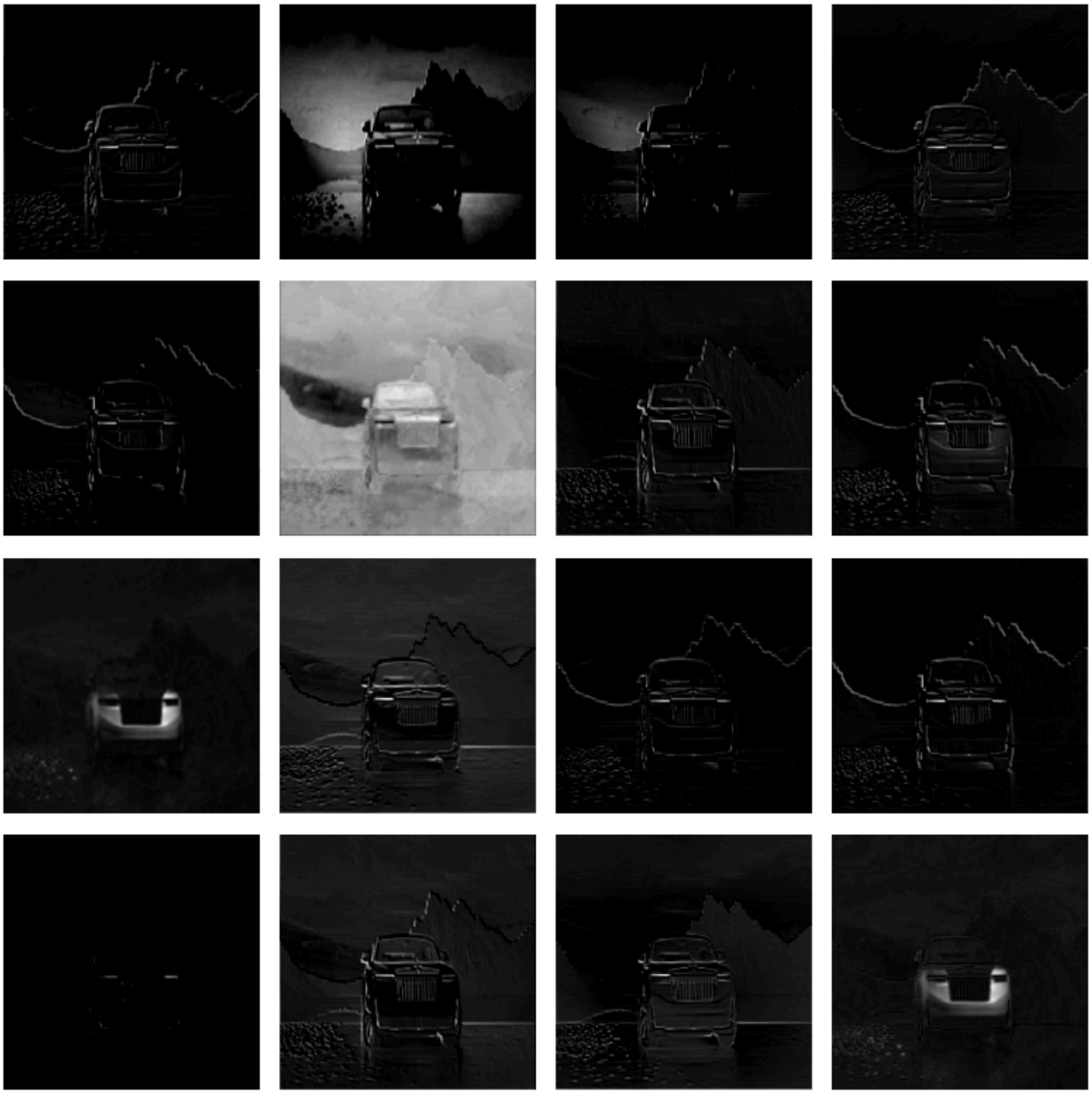
This confirms that convolution filters can be learned effectively using gradient descent.

Deep CNN Feature Visualization (Task 3)

Pretrained models used:

- VGG16
- VGG19

- ResNet50
- ResNet50V2



Feature Maps Across Layers

Early layers (Block1):

- Detect edges
- Gradients

- Simple textures

Mid layers:

- Pattern combinations
- Shapes

Deeper layers:

- Abstract object-level features

Observation:

Feature maps become more abstract and spatially compressed as depth increases.

Layer-wise Filter Count and Duplicate Detection

For VGG19 and ResNet50:

For each Conv2D layer:

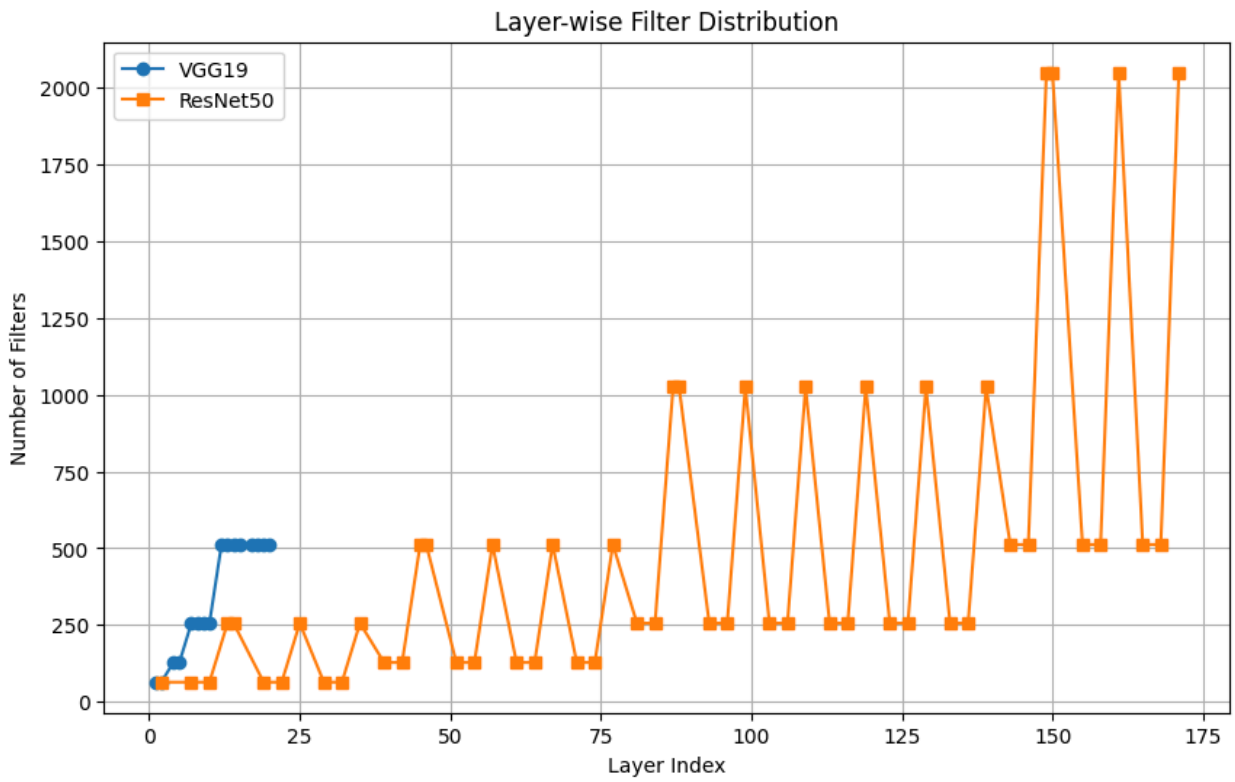
- Extract number of filters
- Plot filter count vs layer index
- Check for duplicate filters

Result:

Duplicate filters found: 0

Interpretation:

- Filters in pretrained models are diverse.
- No redundancy detected.
- Learned representations are unique.



Weight Change Analysis During Inference

Two images were passed through VGG16 and VGG19 using model.predict().

Procedure:

1. Capture original weights
2. Run inference with Image 1
3. Run inference with Image 2
4. Compare weights layer-wise

Result:

Number of changed layers = 0

Explanation:

Inference mode does not update weights because:

- No backward pass
- No optimizer step
- No gradient computation

Thus, pretrained weights remain unchanged.

```
aggregated = (
    table
    .groupby("Model")
    .agg({
        "Changed after Img1?": "sum",
        "Changed after Img2?": "sum"
    })
)

aggregated
```

Model	Changed after Img1?	Changed after Img2?
VGG16	0	0
VGG19	0	0

Layer-wise Learned Filter Weight Changes (Training Scenario)

If training were performed, weight change per layer can be calculated as:

ΔW_l = W_l(after) − W_l(before)

Systematic measurement:

1. Capture weights before training
2. Perform forward + backward pass
3. Capture weights after optimizer step
4. Compute:

- Total changed weights

- Weight increases
- Weight decreases
- Percentage change

If weight decay regularization is used:

$W_new = W - \eta \nabla L - \lambda W$

Where:

η = learning rate

λ = weight decay multiplier

This allows quantifying both weight gain and decay per layer.

Fault-Tolerant CNN Using Checksums (Task 4)

To validate convolution correctness, the following schemes were implemented:

11.1 Batch Checksum

Sum across batch dimension:

$\Sigma \, Conv(X_i, W) = Conv(\Sigma \, X_i, W)$

Result: Verified (True)

11.2 Filter Checksum

Sum across filters:

$\Sigma \, Conv(X, W_j) = Conv(X, \Sigma \, W_j)$

Result: Verified (True)

11.3 Weighted Batch Checksum

Weighted sum:

$\Sigma \, i \cdot Conv(X_i, W) = Conv(\Sigma \, iX_i, W)$

Result: Verified (True)

11.4 Weighted Filter Checksum

$\Sigma\, j \cdot \mathrm{Conv}(X, W_j) = \mathrm{Conv}(X, \Sigma\, jW_j)$

Result: Verified (True)

11.5 Row and Column Checksum

Row reduction:

Sum along height dimension

Column reduction:

Sum along width dimension

Both schemes verified shape consistency and distributive behavior.

12. Observations and Key Findings

1. Convolution is distributive over both input and filter sums.
2. Pretrained models contain no duplicate filters.
3. Early CNN layers capture edges; deeper layers capture abstractions.
4. Inference does not modify model weights.
5. Gradient descent can successfully learn spatial filters.
6. Checksum verification confirms numerical integrity of convolution outputs.

13. Conclusion

The experimental results confirm:

- Correct convolution implementation
- Preservation of numerical integrity
- No weight mutations during inference
- Diverse feature learning in pretrained models